

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

Practical No: 4

Aim: Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

CLI Method:

Code:

```
import rsa

def main():
    print("--- RSA DIGITAL SIGNATURES: CLI ---\n")

    print("Generating RSA Keys...")
    pub_key, priv_key = rsa.newkeys(512)
    print("[+] Keys generated successfully.\n")

    message = input("🗨 Enter the message to sign: ")
    encoded_message = message.encode('utf-8')

    print("\nSigning the message using the Private Key...")
    signature = rsa.sign(encoded_message, priv_key, 'SHA-256')
    print("[+] Message signed successfully. 🔒 ")

    print("\nVerifying signature with the Public Key...")
    try:
        rsa.verify(encoded_message, signature, pub_key)
        print(" VERIFIED: The signature is valid. Message integrity confirmed. ")
    except rsa.VerificationError:
        print(" FAILED: Invalid signature. ")

    print("\n ⚠ Simulating a tampered message in transit...")
    fake_message = (message + " (tampered)").encode('utf-8')
    print(f"Intercepted message reads: {fake_message.decode('utf-8')}")

    print("\nVerifying the tampered message...")
    try:
        rsa.verify(fake_message, signature, pub_key)
        print(" VERIFIED")
    except rsa.VerificationError:
        print(" FAILED: Tampering detected! Signature verification failed. \n")

if __name__ == "__main__":
    main()
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

Output:

```
IDLE Shell 3.146
File Edit Shell Debug Options Window Help
===== RESTART: C:/Users/info/Documents/Practical No 4 Cyber & Information Security T097.py =====
--- RSA DIGITAL SIGNATURES: CLI ---

Generating RSA Keys...
[+] Keys generated successfully.

O Enter the message to sign: 12345

Signing the message using the Private Key...
[+] Message signed successfully. @

Verifying signature with the Public Key...
VERIFIED: The signature is valid. Message integrity confirmed.

^ Simulating a tampered message in transit...
Intercepted message reads: 12345 (tampered)
Verifying the tampered message...
FAILED: Tampering detected! Signature verification failed.

>>>
>>>
>>>

...
>>>

...
>>>

>>>
>>>
>>>
>>>
>>>
>>>
>>>
```

GUI Method:

Code:

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
import rsa
```

```
class DigitalSignatureApp:
```

```
def __init__(self, root):
```

```
self.root = root
```

```
self.root.title("Practical 4: Digital Signature")
```

```
self.root.geometry("450x600")
```

```
self.root.configure(bg="#1e1e2e") # Dark mode background
```

```
# Memory for our keys and signature
```

```
self.pub_key = None
```

```
self.priv_key = None
```

```
self.signature = None
```

```
# --- UI LAYOUT ---
```

tk.Label(

Omith Thilakan

T097

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

```
root, text="RSA Digital Signature",
font=("Helvetica", 18, "bold"), bg="#1e1e2e", fg="#cba6f7"
).pack(pady=(20, 10))

# --- STEP 1: KEYS ---
self.btn_keys = tk.Button(
    root, text="1. Generate Keys", command=self.generate_keys,
    bg="#89b4fa", fg="black", font=("Helvetica", 11, "bold"),
    relief="flat", width=20, pady=5
)
self.btn_keys.pack(pady=10)

self.lbl_status = tk.Label(
    root, text="No keys generated yet.",
    bg="#1e1e2e", fg="#a6adc8", font=("Helvetica", 10, "italic")
)
self.lbl_status.pack()

tk.Frame(root, bg="#313244", height=2, width=350).pack(pady=20)

# --- STEP 2: SIGN ---
tk.Label(
    root, text="Enter Message:",
    bg="#1e1e2e", fg="#f38ba8", font=("Helvetica", 11, "bold")
).pack(pady=5)

self.txt_message = tk.Entry(
    root, width=35, font=("Helvetica", 12),
    bg="#313244", fg="white", insertbackground="white", relief="flat"
)
self.txt_message.pack(pady=5)
# Highlight effect when clicking the text box
self.txt_message.bind("<FocusIn>", lambda e: self.txt_message.config(bg="#45475a"))
self.txt_message.bind("<FocusOut>", lambda e: self.txt_message.config(bg="#313244"))

self.btn_sign = tk.Button(
    root, text="2. Sign Message", command=self.sign_message,
    bg="#a6e3a1", fg="black", font=("Helvetica", 11, "bold"),
    relief="flat", width=20, pady=5
)
self.btn_sign.pack(pady=15)

tk.Frame(root, bg="#313244", height=2, width=350).pack(pady=20)
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

```
# --- STEP 3: VERIFY ---
tk.Label(
    root, text="Verify Message Integrity:",
    bg="#1e1e2e", fg="#f9e2af", font=("Helvetica", 11, "bold")
).pack(pady=5)

self.btn_verify = tk.Button(
    root, text="3. Verify Signature ", command=self.verify_message,
    bg="#f9e2af", fg="black", font=("Helvetica", 11, "bold"),
    relief="flat", width=20, pady=5
)
self.btn_verify.pack(pady=10)

self.lbl_result = tk.Label(
    root, text="", bg="#1e1e2e", fg="ffffff", font=("Helvetica", 13, "bold")
)
self.lbl_result.pack(pady=20)

# --- CRYPTO LOGIC ---
def generate_keys(self):
    # Generate 512-bit RSA keys
    self.pub_key, self.priv_key = rsa.newkeys(512)
    self.lbl_status.config(text="Keys generated successfully. ", fg="#a6e3a1")

def sign_message(self):
    if not self.priv_key:
        messagebox.showwarning("Missing Keys", "Please generate keys first.")
        return

    message = self.txt_message.get()
    if not message:
        messagebox.showwarning("Empty Message", "Please enter a message to sign.")
        return

    encoded_message = message.encode('utf-8')
    # Hash the message using SHA-256 and encrypt the hash with the private key
    self.signature = rsa.sign(encoded_message, self.priv_key, 'SHA-256')

    messagebox.showinfo("Secured", "Message signed successfully. 🔒 ")
    self.lbl_result.config(text="") # Clear previous results

def verify_message(self):
    if not self.signature:
        messagebox.showwarning("Missing Signature", "Please sign a message first.")
```

SHETH LUJ AND SIR MV COLLEGE
Cyber & Information Security

```
return

# Fetch the current text in the entry box to test it
current_message = self.txt_message.get().encode('utf-8')

try:
    # Use public key to decrypt signature and compare against the message hash
    rsa.verify(current_message, self.signature, self.pub_key)
    self.lbl_result.config(text=" VALID: Signature verified.", fg="#a6e3a1")
except rsa.VerificationError:
    self.lbl_result.config(text=" INVALID: Message has been tampered with.", fg="#f38ba8")

if __name__ == "__main__":
    root = tk.Tk()
    app = DigitalSignatureApp(root)
    root.mainloop()
```

Output:

